Topic 1

We want to organize the following gas station and fuel data into an XML structure considering:

a) the organization of the data

b) that the file will have data from several gas stations

c) that each gas station can sell more than one fuel.

The XML you will create should be well-formed (with correct syntax) and should also follow good practices regarding what is encoded as an XML element and what is encoded as an element attribute. For element and property names use the names given below. Avoid creating a shallow tree of depth 3 which is the easy solution. The goal is to use as many features of the XML language as possible, for practicing.

Your XML should have indicative data, both in elements and properties. Give your own prices, making sure they are realistic. Data for three gas stations with 2 fuels in each is enough! The organization/grouping below is indicative – you are not required to follow it.

Check the syntax of your XML with an on-line XML syntax checker.

Generic Data

Geolocation of the user that requests data: **lat**, **long**

Timestamp of the call: **qTimeStamp**

System message to the user (e.g. "Service with not be available on Monday.": **msg**

Gas station Data

Country name and its code: **county**, **countyID**

Municipality and its code: **municipality**, **municipalityID**

Municipal district and its code: **dd**, **ddID**

Gas station address: **address**

Company mame: **companyName**

Gas station coordinates: **lat**, **long**

Distance approximation to the user: **distance**

If it operates all day (24 hours): **h24**

Fuel brand (BP, Shell, etc) and its code: **brand**, **brandID**

Phones: **phone** (from none to two)

Fuel Data

Fuel type code: **fuelTypeID**

Fuel name: **fuelName**

Price per lt: **price**

Timestamp the station owner registered the fuel pprice: **priceTimeStamp**

Topic 2

Based on the fuel.xml file of the previous topic, define a DTD grammar that makes the XML file valid. The grammar should be written inside the XML file, and it should be modified appropriately (in its first line) so that when opening it (e.g. with a browser) a validity check is automatically performed.

Create some problems yourself in the XML data and see if the validation does fail. At the end, pass your XML/DTD through some online validator to see if it is flawless. There are tools to create DTD from XML, but this doesn't work in any context. It is ok to get an initial DTD version, but you must check EVERY line to see if it agrees with the requirements.

Topic 3

Convert the XML file to JSON following good conversion practices (e.g. add a prefix to JSON elements derived from XML attributes, etc.). Use online converters after creating your JSON and compare.

Pay attention to the conversion of multiple XML elements (gas station, fuel elements) – you must use JSON array (there is no other way).

Topic 4

Create a simple HTML page with 2 DIVs, side by side. The one on the left must be 150px wide while the other must occupy the rest available width. The page should contain:

1. A button on the left div to load (with an AJAX call) the XML file and display some elements and attributes on the right div (stationID, distance, address, value of h24, phone as well as all the data of the fuel elements).
2. A button on the left div to load (with an AJAX call) the JSON file and display some data on the right div (stationID, distance, address, value of h24, phone as well as all the fuel data).
3. A function to clean up the right div, that will be called automatically in script 1 and 2 above, as well as when a 3rd [Clear] button is pressed (you will put that on the left div, as well).

The association of the function with the click on the 3rd button should be done in a dynamic way. For cases/buttons #1 and #2 use the onclick attribute (static JS assignment). Both approaches are explained here: https://www.freecodecamp.org/news/html-button-onclick-javascript-click-event-tutorial/

Deliverables: a) a fuel.xml file with the DTD grammar included,
b) a JSON file with the equivalent data (not the grammar) of the XML
c) a single page app demonstrating Topic 4.

Deadline: one month after the announcement

Please ask for help if you are in trouble.